

LYRA-CPP TX EXECUTION PLAN — Step 14 → Phase 3

Branch: tx-rebuild

Reference tree: D:\sdrprojects\OpenHPSDR-Thetis-2.10.3.13\Project Files\Source\

- ChannelMaster\networkproto1.c — HL2 P1 wire layer
- ChannelMaster\network.c — sendPacket + WriteUDPFrame + KeepAliveLoop
- ChannelMaster\network.h — _radionet struct + globals
- ChannelMaster\netInterface.c — create_rnet, thread spawn handshake
- ChannelMaster\cmaster.c, cmsetup.c — WDSP TXA channel setup
- Console\radio.cs — high-level WDSP TXA setter ordering
- Console\console.cs — MOX state machine (chkMOX_CheckedChanged2)
- wdsp*.c — WDSP DSP source (port-with-attribution per CLAUDE.md §2)

Created: 2026-06-06 by operator directive.

Owner: N8SDR. Co-author trailer on every commit: Co-Authored-By: Claude Opus 4.7 (1M context)

■ LOCKED OPERATING RULES — READ AT START OF EVERY SESSION

These rules supersede any prior pattern. Violations cost real session time and operator trust. Every session begins by reading this section.

Rule 1 — DO AS THE REFERENCE DOES, PERIOD. NO PATCHING.

The reference IS the spec. Lyra mirrors the reference byte-for-byte and structure-for-structure within the explicit C↔C++23 idiom translations (Rule 5). Anything Lyra does that the reference does not → PATCH → reverts. Anything the reference does that Lyra omits → MISSING → fix or explicit operator-acknowledged deferral with file:line citation.

Rule 2 — NO PORTING FROM PYTHON-LYRA FOR TX. EVER.

Python-Lyra TX was broken. It is NOT a valid source for lyra-cpp TX behavior. References to lyra-Python §15.xx patterns are HISTORICAL CONTEXT only and may NOT be the basis for any code decision. If the only design source for an item is lyra-Python, that item is **STOP-AND-ASK** territory.

Rule 3 — READ REFERENCE TWICE BEFORE WRITING ANY CODE.

First read: understand what the reference does. Second read: verify the first reading against the actual bytes/lines (catches misremembering). Write code only AFTER both reads. Quote the exact reference file:line in the code comment so a future reader can verify.

Rule 4 — IF UNSURE, STOP AND TALK. NO "PATCH AND CALL IT FIXED."

If the reference body is ambiguous, missing, or contradicts another reference site → STOP. Surface to operator. Operator decides. No more inventing a "reasonable interpretation" that turns out to be wrong on bench.

Rule 5 — ACCEPTABLE C↔C++23 IDIOM TRANSLATIONS (allow-list).

ONLY the following are auto-acceptable as "C-IDIOM" — anything else is a PATCH unless operator-signed:

- malloc/calloc → std::vector::assign or new T()
- memset(p,0,N) → std::fill_n or vector default
- CRITICAL_SECTION + EnterCriticalSection/LeaveCriticalSection → std::mutex + std::lock_guard
- CreateSemaphore + WaitForSingleObject + ReleaseSemaphore → std::counting_semaphore<1>
- WaitForMultipleObjects(2, ..., TRUE, INFINITE) (wait-all on max-count-1 semaphores) → std::condition_variable + paired-bool predicate
- HANDLE _beginthreadex → std::thread / std::jthread

- `Sleep(N) → std::this_thread::sleep_for(Nms)`
- `unsigned char ptr[N]` stack buffer → `std::array` stack
- UB signed-int byte aliasing → portable shifts via `uint32` + signed cast
- C-style unscoped enum + shadow var → `enum class` + distinctly-named global
- C-struct → C++ class with embedded members (semantics preserved)
- C free function → C++ free function in namespace (no class wrapping unless operator-signed)
- File-scope global `unsigned int x` (non-atomic) → TU-scope `std::uint32_t x` or `atomic`; mark in comment which.

Rule 6 — NOTHING GETS CHECKED OFF WITHOUT 2 INDEPENDENT AUDITS.

A checkbox in this file is marked complete ONLY after:

- **Audit #1** — Claude reads reference + Lyra side-by-side line-by-line, produces a CLEAN/PATCH table per the established methodology, file:line cited on BOTH sides.
- **Audit #2** — Independent re-verification. May be: (a) a fresh background agent, (b) operator bench-verification, (c) Claude re-reads in a fresh session with no prior context. Must produce its OWN file:line-cited table.
- BOTH audits must report CLEAN (or operator-signed-DEFERRED) for the item before the checkbox is marked.

Rule 7 — RX-BENCH-PASSING ≠ REFERENCE-FAITHFUL.

Bench validation is NECESSARY but NOT SUFFICIENT. A stage cannot be marked complete because RX still works. Stage requires BOTH bench-pass AND reference-parity audit pass.

Rule 8 — §3.9 OPERATOR-EMPIRICAL DISCIPLINE.

Bench-verified gateway-RTL divergences (e.g. HL2+ ak4951v4 20-bit seq mask, telemetry slot re-map) outrank source-inference. BUT they must be:

- Documented in code with explicit RTL/file:line provenance comments
- Listed in this file's **§3.9 Operator-Signed Divergences** registry
- Operator-signed before they ship (no Claude-decided "I think this is per §3.9")

Rule 9 — BUILD + RULE-2 GREP CLEAN PER COMMIT.

Every commit:

- `cmd //c '._b.bat'` exits 0 with zero error `C[0-9]+ / fatal error`
- Rule 2 forbidden tokens (Thetis, thetis, PowersDR, powersdr, Console.cs, OpenHPSDR, openhpsdr) absent from shipped code (in `src/`, `qml/`); allowed in docs/
- Co-author trailer present
- Backup bundle `_backups/lyra- - .bundle` if the commit is bench-gate-eligible

Rule 10 — COMMIT IN STAGES SMALL ENOUGH TO REVERT.

Each stage of this plan = its own commit. No multi-stage mega-commits. Each commit revertible on bench failure without losing the prior stage's progress.

■ REFERENCE-PARITY VERIFICATION PROTOCOL (mandatory per item)

For EVERY non-trivial code change in Step 14 or Phase 3:

Pre-write

1. **Locate reference body.** Grep for the function name in the reference tree. Cite file:line. If not found → STOP-AND-ASK.
2. **First read.** Read the reference body in full. Note what it does in working memory.
3. **Second read.** Re-read the same body. Verify the first reading is accurate. Note any quirks/defects to preserve per Rule 24.

4. **Read Lyra side.** Read the current Lyra equivalent in full (if it exists).
5. **Build the parity table** before any code is written. Columns: Lyra (file:line) | Lyra quoted | Reference (file:line) | Reference quoted | Verdict | Notes. Verdicts per Rule 6.
6. **If the table shows anything not in the C-IDiom allow-list (Rule 5) without operator sign-off → STOP-AND-ASK.**

Write

7. Write Lyra code that matches the reference behavior within Rule-5 idioms. Comments cite reference file:line.

Post-write

8. Build clean per Rule 9.
9. Rule 2 grep clean.
10. Run **Audit #1** — re-read both sides, re-build the parity table from scratch, confirm CLEAN.
11. Run **Audit #2** — independent re-verification (fresh agent OR fresh session OR operator bench).
12. Commit with descriptive message + co-author trailer + reference cites in body.
13. Mark the checkbox in this file with: [x] {date} {commit hash} audit1=PASS audit2=PASS.

Bench-gate stages (Stage 2 and Stage 8 of Step 14, plus any explicitly marked)

14. Operator bench-validates per the stage's bench-gate criteria.
15. Operator sign-off recorded inline next to the checkbox: OPERATOR-SIGNED {date}.

■ DAILY SESSION-START RITUAL

At the start of every session, Claude reads this file in this order:

1. **LOCKED OPERATING RULES** (Section above).
2. **§3.9 OPERATOR-SIGNED DIVERGENCES REGISTRY** (Section below).
3. **PROGRESS TRACKING — STEP 14** (current stage status).
4. **OPEN STOP-AND-ASK ITEMS** (if any).
5. **LAST-COMMIT LOG** (last 3-5 commits on tx-rebuild).

Then asks the operator: "What's the focus this session?" before proposing any code.

Sync the operator-readable copies (DOCX + PDF) each session-start

After any edit to this EXECUTION_PLAN.md, regenerate the operator-readable copies:

```
py -3.14 tools\sync_execution_plan.py --pdf
```

Produces:

- docs/EXECUTION_PLAN.docx — Word-readable mirror, regenerated from the MD source. Stamped with generation timestamp in the header.
- docs/EXECUTION_PLAN.pdf — paginated PDF mirror. Stamped with generation timestamp + page numbers in the footer.

EXECUTION_PLAN.md is the **source of truth**. The DOCX + PDF are auto-generated read-only views — never edit them directly. If they drift from the MD, re-run the sync script.

The sync script is committed at tools/sync_execution_plan.py. Requires Python with python-docx + reportlab (Python 3.14 on this machine has both).

■ OPEN STOP-AND-ASK ITEMS (Claude blocks here)

(none currently — populate as they arise)

■ §3.9 OPERATOR-SIGNED DIVERGENCES REGISTRY

Per Rule 8 — bench-verified gateway-RTL divergences that explicitly deviate from reference source. Each entry needs operator sign-off date.

#	Divergence	Reference says	Lyra (with RTL/file:line provenance)	Operator-signed?
§3.9-1	EP6 seq counter width	32-bit (networkproto1.c:191-194)	20-bit mask kSeqMask20 = 0x000FFFFF (HL2+ ak4951v4 RTL: logic [19:0] ep6_seq_no per hl2_stream.cpp:1091-1118 comment block) — operator bench-verified the gateway wraps every ~3:27 min	PENDING SIGN-OFF
§3.9-2	Telemetry slot semantics — case 0x08 C1:C2	exciter_power (networkproto1.c:506-507)	temp on HL2+ ak4951v4 — operator bench-verified	PENDING SIGN-OFF
§3.9-3	Telemetry slot semantics — case 0x10 C3:C4	user_adc0 ("AIN3 MKII PA Volts" per Thetis HL2 commentary)	PA current — operator bench-verified PA = 1.76 A at full tune vs Thetis 1.8 A anchor (CLAUDE.md §15.26 Correction-3)	PENDING SIGN-OFF
§3.9-4	Telemetry slot semantics — case 0x18	user_adc1 (C1:C2) + supply_volts (C3:C4) per networkproto1.c:516-517	dead/junk on HL2+ ak4951v4 — operator bench-verified (CLAUDE.md §15.26)	PENDING SIGN-OFF
§3.9-5	MoxEdgeFade for SSB	WDSP TXAuslewCheck at wdsp/TXA.c:819-824 returns 0 for SSB → no WDSP envelope at all for SSB	Lyra-native cos ² shim in src/mox_edge_fade.cpp (336 LOC) — necessary because reference doesn't address SSB envelopes; mid-fade reversal continuity is Lyra-native math	PENDING SIGN-OFF

Operator: when ready to sign off on an item, replace **PENDING SIGN-OFF** with **SIGNED {YYYY-MM-DD}**.

■ PROGRESS TRACKING — STEP 14 (Wire-Layer Migration)

Reference: docs/architecture/STEP14_PLAN.md (Stages 1-10 spec).

Stage 1 — Wire-Layer Singleton + Bind, Wire-INERT

- SHIPPED earlier 2026-06-06 (compile-and-link only; RX bench passed)
- Audit #1: PASS (today's wire-layer round)
- Audit #2: PASS (operator RX bench-verified)
- Status: ■ COMPLETE

Stage 1.5 — Wire-Layer Reference-Parity Fix Pass (audit cleanup, today's work)

- Commit A — drop FIX 8 hwriteThreadInitSem vestige (Ep2SendThread.cpp)
- Commit B — Ep6RecvThread Round-2 carryovers (MetisReadDirect local-stack 1074-byte buffer + rcvpktp1 lock + reference-verbatim seq check + drop g_seq_seen + drop WSACloseEvent on stop + drop duplicate metis_wire_bind)
- Commit C — cleanup deletes (drop AvRevertMmThreadCharacteristics both threads + drop ForceCandC defensive null guards + drop write_main_loop_hl2 defensive asserts + drop scheduler prologue C1..C4 re-zero + drop 19 per-case assert(prn !=

nullptr) defensive guards + simplify setters to pure single-line writes + add bandwidth_monitor_reset() stub anchor)

- Commit D — OutboundRing reference-parity (fix initial state lr/iq_consumed=false; producer order memcpy → set_ready → notify → wait; drop outbound_unblock + outbound_stop; bounded wait_for(100ms) poll)
- Commit E — MetisFrame 2-layer MetisWriteFrame → send_packet restored (per-call sockaddr from g_metis_addr_be + prn->base_outbound_port; prn->sndpkt lock around sendto; bandwidth_monitor_out anchor; sendto failed error log)
- Commit F (cosmetic) — mb shadow warning fix in hl2_stream.cpp::composeCC case-10
- Audit #1: PASS (4-agent wire-layer audit confirmed 196 CLEAN comparisons + all PATCH-class findings resolved)
- Audit #2: PENDING (operator bench-verify — wire-layer tree still INERT, so bench-verify happens via Stage 2 first-wire test)
- **Status:** ■ COMPLETE (pending Audit #2 via Stage 2 bench-gate)

Stage 2 — ForceCandC::prime(3, tx_freq, rx_freq) *Insert, Wire-LIVE on First HL2 Talk*

Bench-critical commit. First time new wire layer talks to real HL2 hardware.

Reference order (verbatim per networkproto1.c:427-434 + netInterface.c:50, 60-66):

```
HL2Stream::open():
  1. UDP socket open + state clear (existing)
  2. prn = create_rnet(); metis_wire_bind(); outbound_init(); ← Stage 1 (done)
  3. SendStartToMetis equivalent — buildControlPacket(true) + sendto on socket_
    (hoisted from old txWorkerLoop; mirrors netInterface.c:50)
  4. Ep6RecvThread::start(socket_) ■■■■ caller blocks on init-sem ■■■■
    ■
In Ep6RecvThread::run_loop():
  a. allocate FPGA buffers (calloc(1024)) [networkproto1.c:427] ■
  b. ForceCandCFrame(3) [networkproto1.c:430] ■
  c. WSACreateEvent [networkproto1.c:433] ■
  d. WSAEventSelect(socket, event, FD_READ) [networkproto1.c:434] ■
  e. init_sem.release() [netInterface.c:61-63]■
  f. while (io_keep_running) { WSAWaitForMultipleEvents + dispatch }
  5. caller unblocks; seed `txSeq_` from `metis_out_seq_num()`; spawn TX worker
```

STOP-AND-ASK first: the plan's "stage-boundary call" — does Stage 2 spawn Ep6RecvThread as the LIVE rcv path (collapsing Stage 3+6 into Stage 2), OR does it leave the old rxworker_jthread live and Ep6 only does priming+park-until-Stage-6? Operator must answer before Stage 2 starts.

Per-item checklist:

- **Stage-boundary call decision** (operator answers question above)
- Pre-write: read networkproto1.c:427-434 twice; verify against today's Ep6RecvThread::run_loop() body
- Pre-write: read netInterface.c:50, 60-66 twice; build parity table for the SendStartToMetis hoist
- Add init-sem release point in Ep6RecvThread::run_loop AFTER WSAEventSelect AND force_candc_frame(3) complete
- Add init-sem acquire in HL2Stream::open() after Ep6RecvThread::start() returns
- Hoist buildControlPacket(true) + ::sendto from txWorkerLoop:2451-2461 into HL2Stream::open() BEFORE the Ep6 spawn
- Delete the open-time START send from txWorkerLoop
- Seed txSeq_ = lyra::wire::metis_out_seq_num() AFTER init-sem releases, BEFORE txworker_ spawns
- Build clean per Rule 9
- Rule 2 grep clean
- Audit #1: []
- Audit #2: []
- Operator bench-gate per STEP14_PLAN.md §Stage 2 (cold open → 6 priming datagrams emit → RX still works on known signal → no abnormal seq/framing errors over 5 min)
- OPERATOR-SIGNED {date}

• **Status:** ■ NOT STARTED

Stage 3 — Stand Up Ep6RecvThread In Parallel, Mute Sinks, Read-Only Compare

(Folds-in only if Stage 2 did NOT collapse this in.)

Per-item checklist:

- Pre-write: re-read Ep6RecvThread::process_datagram body twice
- Expose Ep6RecvThread::process_datagram() publicly
- Call from rxworkerLoop AFTER existing validation, BEFORE old iqSink_ dispatch
- **AUDIT FOLD §3.9-1:** port 20-bit seq mask into Ep6RecvThread (only after operator signs off §3.9-1 above)
- Verify Ep6's start() machinery stays dormant in this stage (no second recvfrom on shared socket)
- Build + Rule 2 + Audit #1 + Audit #2
- Operator bench-gate: A/B comparator — seq error rates, per-DDC IQ, mic harvest, telemetry all match
- OPERATOR-SIGNED {date}

• **Status:** ■ NOT STARTED

Stage 4 — Migrate First Consumer (WDSP IQ Sink) Off iqSink_ Onto Router::register_sink()

Per-item checklist:

- Pre-write: re-read Router + xrouter + twist reference bodies twice
- Register WDSP RX1 IQ sink via Router::register_sink() on Ep6RecvThread
- HL2Stream's iqSink_ dispatch made conditional (if (!routerActive_))
- Build + Rule 2 + Audit #1 + Audit #2
- Operator bench-gate: RX1 audio A/B vs HEAD
- OPERATOR-SIGNED {date}

• **Status:** ■ NOT STARTED

Stage 5 — Migrate Mic + Telemetry + PTT-In Consumers Off HL2Stream Onto Ep6RecvThread sinks

Per-item checklist:

- Pre-write: re-read networkproto1.c:478-525 (telemetry decode) twice
- **AUDIT FOLD §3.9-2 / §3.9-3 / §3.9-4:** port telemetry slot re-map into Ep6RecvThread::decode_status_header (ONLY after operator signs off §3.9-2/3/4 above)
- **AUDIT FOLD (real ungated):**
 - Verify micDecimationCount reset at Ep6RecvThread run_loop entry (today's Commit B should have this)
 - Verify seqErrors reset at run_loop entry
 - Verify first-frame seq check is reference-verbatim (today's Commit B)
 - Verify ptt_in/dash_in/dot_in shadows preserved per Rule 24 (reference defects intact)
 - Add case 0x20 per-ADC overload decode (currently missing in Ep6RecvThread::decode_status_header)
- **AUDIT FOLD HW-PTT forwarder:** Lyra-native opt-in routed via Ep6 sink (preserve §15.26 RESOLVED-CORRECTION hwPttEnabled_ gate, default OFF)
- Mic source consumer routed via Ep6RecvThread::set_mic_sink
- Telemetry consumer routed via Ep6RecvThread::set_telemetry_sink
- Build + Rule 2 + Audit #1 + Audit #2

- Operator bench-gate: PA-current readout matches §15.26 last-known-good (1.76A at full tune); foot-switch (if opted in); mic feeds DSP correctly
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 6 — Activate `Ep6RecvThread::start()` As Its Own `std::thread`, Delete `HL2Stream::rxWorkerLoop`

Per-item checklist:

- Pre-write: re-read `MetisReadThreadMainLoop_HL2` start-to-end twice
 - `Ep6RecvThread::run_loop` becomes the live recv path
 - Delete `HL2Stream::rxWorkerLoop` body
 - Delete Stage 3's inline tap from old `rxWorkerLoop` (now dead code)
 - Build + Rule 2 + Audit #1 + Audit #2
 - Operator bench-gate: 30-min soak + 5× stop+restart cycles, no regressions
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 7 — Migrate TX-Side Setters Onto `FrameComposer` Free Functions

Per-item checklist:

- Pre-write: re-read `writeMainLoop_HL2:869-1201` setter call sites twice
 - `HL2Stream` setters (`setRx1FreqHz`, `setTxFreqHz`, `setLnaGainDb`, `setTxStepAttnDb`, `setPaOn`, `setMicBoost`, etc.) dual-write: existing atomic AND `lyra::wire::set_*` free function
 - **AUDIT FOLD:** `HL2Stream::set_pa_on` → `setApolloTuner (0x08) + ApolloFilt (0x04)` globals correctly so `FrameComposer` case-10 emits the §15.26 PA-on C2 = 0x4C
 - **AUDIT FOLD:** `HL2Stream::setLnaGainDb` write path matches `FrameComposer` case-11 RX-branch wire byte (0x40 | ((g+12) & 0x3F))
 - **AUDIT FOLD:** `HL2Stream::setTxStepAttnDb` write path matches `FrameComposer` setter (31 - signed_db)
 - Build + Rule 2 + Audit #1 + Audit #2
 - No bench gate (inert debug read-back only). Verify `FrameComposer` reads match `HL2Stream` atomics via diagnostic dump.
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 8 — Migrate `EP2 Egress` Onto `Ep2SendThread + write_main_loop_hl2()`. SECOND HL2 BENCH GATE.

Bench-critical commit. Load-bearing TX-side migration. Resolves all 19 LIVE-A composeCC PATCHes + all 19 LIVE-B txWorkerLoop PATCHes in one cut.

Per-item checklist:

- Pre-write: re-read `sendProtocol1Samples:1204-1267` twice
- Pre-write: re-read `MetisWriteFrame:216-237 + sendPacket@network.c:1382-1402` twice
- `HL2Stream::open` spawns `Ep2SendThread` instead of `txworker_jthread`
- LRIQ samples flow: `HL2Stream` audio path → `outbound_push_lr/iq` → `Ep2SendThread` → `MetisFrame`
- **AUDIT FOLD MoxEdgeFade:** verify cos² envelope output reaches `OutboundRing` IQ buffer (adapter if needed)
- **AUDIT FOLD §15.20 host TX-timeout:** verify `lyra-cpp` equivalent exists; add if missing

- **AUDIT FOLD §3.9-5:** MoxEdgeFade Lyra-native shim acknowledged per §3.9-5 sign-off
 - Delete `HL2Stream::txWorkerLoop` body
 - Delete `HL2Stream::composeCC` body (replaced by `FrameComposer::write_main_loop_hl2`)
 - Build + Rule 2 + Audit #1 + Audit #2
 - **Operator bench-gate:** first-RF + PA-current readout MUST match §15.26 last-known-good (~1.76A at full tune, ~5W on Palstar). 30-min soak. 5× stop+restart cycles. No regressions.
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 9 — Delete Stage-7 Dual-Write + Retire HL2Stream-side TX Atomics + Unused State

Per-item checklist:

- Pre-write: enumerate all `HL2Stream` TX atomics that became dead after Stage 8
 - Delete dual-write paths
 - Delete dead atomics
 - Delete `HL2Stream::queueTxAudio`, `txIqSource_`, etc. per `STEP14_PLAN.md`
 - Build + Rule 2 + Audit #1 + Audit #2
 - Operator bench-gate: functional regression check (key, hear sidetone if any, telemetry, RX continues)
 - OPERATOR-SIGNED {date}
 - **Status:** ■ NOT STARTED
-

Stage 10 — Final Cleanup + Doc Sweep

Per-item checklist:

- Delete unused `destStorage_` member in `HL2Stream.h`
 - Drop the `hwriteThreadInitSem` member from `RadioNet.h` IF strict reference parity requires (reference DECLARES it but never uses; can stay for parity)
 - Update `CLAUDE.md` to reflect Step 14 complete + Phase 3 next
 - Update `docs/architecture/PARITY_CHECKPOINTS.md`
 - Sweep Rule 2 forbidden tokens project-wide (Task #73)
 - Final build + Rule 2
 - No bench gate (doc + cleanup only)
 - **Status:** ■ NOT STARTED
-

■ PROGRESS TRACKING — PHASE 3 (after Step 14)

Phase 3 begins ONLY after Step 14 Stage 10 is OPERATOR-SIGNED.

Phase 3.1 — `src/tx/TrSequencing.h` populate fields

- Pre-write: re-read `CLAUDE.md` §15.26 LOCKED reconcile + Thetis DB export values
- Add fields: `rf_delay_ms`, `mox_delay_ms`, `space_mox_delay_ms`, `ptt_out_delay_ms`, `key_up_delay_ms`
- Defaults: `rf_delay=50` (operator-tunable 1..75 hot-switch range), `mox_delay=10`, `space_mox_delay=13`, `ptt_out_delay=20`, `key_up_delay=10`
- Build + Audit #1 + Audit #2
- **Status:** ■ NOT STARTED

Phase 3.2 — `src/tx/KeydownSequence.cpp` *implement per §15.25 ground truth*

- Pre-write: re-read `console.cs::chkMOX_CheckedChanged2:30058+` twice
- Implement: RX-DSP stop (`SetChannelState(id(0,0),0,1) dmode=1 BLOCKING flush`) → ATT-on-TX → UpdateDDCs equivalent → MOX-bit set → if !CW: `rf_delay` → TX-DSP start (`SetChannelState(id(1,0),1,0)`)
- Build + Audit #1 + Audit #2
- Bench-gate: dummy-load TX, scope keydown order matches reference
- Status: ■ NOT STARTED

Phase 3.3 — `src/tx/KeyupSequence.cpp` *implement per §15.25 ground truth*

- Pre-write: re-read `console.cs::chkMOX_CheckedChanged2:30350+` twice
- Implement: TX-DSP off (`SetChannelState(id(1,0),0,1) dmode=1 BLOCKING TX downslew flush`) → `mox_delay` sleep → clear MOX bit → `ptt_out_delay` sleep → RX-DSP restart
- Build + Audit #1 + Audit #2
- Bench-gate: dummy-load TX, scope keyup order matches reference (no key-click/splatter)
- Status: ■ NOT STARTED

Phase 3.4 — `src/tx/AttOnTxPolicy.cpp` *implement*

- Pre-write: re-read `console.cs:30293-30327 (keydown) + :30391-30410 (keyup)` twice
- Implement: keydown save per-band RX1 step-att + force-31 (when PS-A off or CW); keyup restore
- Build + Audit #1 + Audit #2
- Bench-gate: keying does NOT overload RX ADC (panadapter doesn't go wide on TX)
- Status: ■ NOT STARTED

Phase 3.5 — `src/tx/PttFsm.cpp` *implement body*

- Pre-write: re-read CLAUDE.md §15.25 FSM ground truth (PttSource set + resolver pattern)
- Implement: PttSource enum, `_resolve()` precedence MOX_TX > CW_TX > VOX_TX > TUN_TX > RX, source-set funnel
- Build + Audit #1 + Audit #2
- Bench-gate: SW-MOX + HW-PTT share state correctly; no phantom keying
- Status: ■ NOT STARTED

Phase 3.6 — `src/wdsp/TxChannel.cpp` *implement body*

- Pre-write: re-read `wdsp/TXA.c::create_txa + xtxa` execution order twice
- Pre-write: re-read `cmaster.c::XCMSetTX*` family twice
- Implement: OpenChannel (type=1) + initial setter sequence per reference `_apply_init_setters` pattern (avoiding the §15.23 SetTXABandpassRun trap that hit lyra-Python)
- Build + Audit #1 + Audit #2
- Bench-gate: TUN tone produces clean carrier; SSB voice produces correct sideband
- Status: ■ NOT STARTED

■ VERIFICATION LOG

Per-stage record of audit runs + operator sign-offs. Newest at top.

Stage	Date	Audit#1 result	Audit#2 result	Operator bench	Operator signed
1	2026-06-06	PASS (today's wire-layer round)	PASS (operator RX bench)	RX clean	✓
1.5 (5 fix commits A-E)	2026-06-06	PASS (4-agent wire-layer audit, 196 CLEAN)	PENDING (gates via Stage 2 first-wire)	—	—

■ NOTES FOR FUTURE SESSIONS

- This file IS the source of truth for current state. Read top-to-bottom at session start.
- Any item with **PENDING SIGN-OFF** blocks the stage that depends on it. Surface immediately.
- §3.9 items: when operator signs off, edit this file with the date + commit hash where the divergence ships, NOT in the §3.9 registry only.
- If Claude reads a reference body and finds Lyra deviating in a way the existing plan doesn't address → STOP-AND-ASK item added to the "OPEN STOP-AND-ASK" section above, do NOT proceed.
- Backup bundle naming: `_backups/lyra-YYYY-MM-DD-stage{N}.bundle`.
- Commit message format: `[Stage {N}].[sub]] {title}\n\nReference: {file:line}\nAudit: #1=PASS #2={result}\n\nCo-Authored-By: Claude Opus 4.7 (1M context) .`

End of EXECUTION_PLAN.md. Status as of 2026-06-06 EOD: Stage 1.5 complete (5 fix commits shipped, build clean, Rule 2 grep clean); Stage 2 unblocked; awaiting operator stage-boundary call + §3.9 sign-offs before Stage 2 starts.